Computer Vision Engineer — Home Assignment

Deliverable format: Public or private Git repository + a short written report (PDF or Markdown)
Submission deadline: 7 days from receipt

---

1. Context

At AeroSentry we build interceptor drones. One of the core perception problems we face is reliably confirming, from an interceptor's onboard camera, that a *flying fixed-wing aircraft* is present in the scene. The detector you build will not run on clean lab data — it will run on shaky, motion-blurred, sometimes overexposed footage captured from a moving interceptor, and it will be the trigger for downstream tracking and engagement logic.

False positives are expensive. A parked plane on a tarmac, an aircraft on a billboard, a model airplane on a shelf, or a plane-shaped cloud must **not** trigger a detection. Missed detections of an actual airborne target are equally expensive.

The eventual deployment target is a **Jetson Orin Nano (8 GB)**. Your design choices should reflect that constraint from day one — not as an afterthought.

You are not expected to ship a production system in 1–2 days. You **are** expected to demonstrate the judgment of an engineer who has shipped one before.

---

2. What we provide

The download link contains:

https://drive.google.com/drive/folders/17lAv6XySpEC74OgOp2vNngNBmqrzD2zf?usp=sharing

- **`fixed wing UAV dataset.v2i.yolov11.zip`** — the labeled dataset of fixed-wing UAVs in YOLOv11 format (`data.yaml`, `train/`, `valid/`, `test/`, with `images/` and `labels/` subfolders in each split).
- **`fixed wing UAV dataset.v2i.coco.zip`** — the same dataset exported in COCO format (`annotations/` JSON files + `images/` directories per split).
- **`Test Footage`** — a small set of test video clips representative of the deployment distribution. **Do not train on these.** They exist for qualitative inspection and final evaluation only.

Use whichever format suits your chosen model. You may also convert to any other format as needed.

The provided dataset is your starting point, not necessarily your finishing point. You may augment it with public data (COCO `airplane`, FGVC-Aircraft, Open Images, etc.) if you can justify why doing so improves robustness. If you augment, document what you added and why.

---

3. The Task

Design and implement a prototype detection pipeline that, given a video stream or sequence of frames, outputs per-frame detections of the form:

```
{ "frame_id": int, "bbox": [x1, y1, x2, y2], "score": float, "class": "airborne_fixed_wing" }
```

The pipeline must address four areas. Treat them as equally weighted.

3.1 Model selection, training & system architecture

- **Pick a detector architecture** (YOLO family, RT-DETR, RTMDet, two-stage, custom, etc.) and **justify the choice** in your report. Your justification should address: accuracy ceiling on this dataset, fit to the Orin Nano deployment envelope, ecosystem maturity (export tooling, documentation, support), and any tradeoffs you accepted.
- Train your chosen model on the provided dataset. Document hyperparameters, augmentations, train/val split discipline, epochs, early stopping, and why you made those choices.
- Define and implement the end-to-end pipeline: frame ingest → preprocessing → detection → false-positive suppression → output. Make the data contracts between stages explicit.
- The architecture should be modular enough that a teammate could swap the detector backbone without touching the FP-suppression stage.

3.2 False-positive reduction

A detector trained only on positive examples will fire on plane-shaped objects that aren't airborne fixed-wing aircraft. Your pipeline must distinguish **airborne fixed-wing UAVs** from plane-like distractors. At minimum, address:

- Grounded aircraft (parked, taxiing, on runways)
- Aircraft in 2D media (billboards, posters, screens, printed materials)
- Toys, models, and scale replicas

- Sky-region distractors that are not fixed-wing UAVs (birds, clouds, kites, multirotor drones, balloons)

You decide the method — sky-region segmentation, motion/parallax cues, temporal consistency, a second-stage classifier, geometric priors, multi-frame voting, or a combination. **Justify the choice.** A simple approach you can defend is better than an elaborate one you cannot.

Note: the provided training dataset likely does not include explicit hard negatives. Part of the assignment is recognizing this and proposing how you'd address it — whether by mining hard negatives from public sources, generating synthetic negatives, leveraging the background class implicitly, or designing the FP-suppression stage as a separate module that doesn't rely on the detector having seen distractors during training.

3.3 Edge optimization for Jetson Orin Nano

You do **not** need access to a Jetson to complete this assignment. You **do** need to make and defend choices that are realistic for the platform:

- Model size and input resolution given Orin Nano's compute budget (~40 TOPS INT8 for the 8 GB variant, ~half that in FP16; no DLAs)
- Quantization strategy (FP16 vs INT8, PTQ vs QAT) and the accuracy/latency tradeoff you expect
- Export path: PyTorch → ONNX → TensorRT (or your chosen alternative), with the gotchas you anticipate (NMS export, dynamic shapes, calibration data selection for INT8, unsupported ops in your chosen architecture)
- Memory budget and how the pipeline shares it with downstream consumers (tracker, comms, etc.)
- Target end-to-end latency and FPS, and how you'd measure them

You may simulate the deployment on x86 with ONNX Runtime or TensorRT, and **project** Jetson numbers from published benchmarks. Be explicit about what is measured vs projected.

3.4 Evaluation & metrics

Define how you would know this system works. Concretely:

- How is the provided dataset split? Is the default `valid/` set sufficient, or do you need a different held-out distribution? Justify.
- Which metrics matter and why?
- How do you evaluate the FP-suppression stage *independently* of the detector?
- How would you evaluate latency and throughput on Orin Nano specifically?
- Run your evaluation on the dataset's `test/` split and on the provided `test_footage/` clips. Report numbers honestly and discuss what they do and do not tell you.

---

4. Deliverables

A single Git repository containing:

1. **Source code** — runnable from a clean environment with documented setup. A `Dockerfile` or `requirements.txt` + a `make run` / `python run.py` entry point is required. Include the trained model weights or a script that produces them deterministically.
2. **A short written report (3–6 pages)** covering:
   - Architecture diagram and a walk-through of the pipeline
   - Model selection rationale, training setup, and training-curve diagnostics
   - The FP-reduction approach and why you chose it over alternatives you considered
   - The Jetson Orin Nano deployment plan: precision, expected latency, memory, and the risks you anticipate
   - Your evaluation setup, metrics, results on both `test/` and `test_footage/`, and an honest discussion of what's missing
   - A "what I'd do with another week" section
3. **A demo** — either a short video, a set of qualitative result images on the provided test footage, or a script that reproduces your evaluation numbers. Whatever lets us see the system run.

---

5. Ground rules

- **Use any tools you'd use on the job**, including LLM assistants. We care about the engineering decisions and your ability to defend them in the follow-up interview, not about whether you typed every line.
- **Be explicit about what you didn't do** and why. "I would have done X but it didn't fit in the time budget, and here's how I'd approach it" is a strong answer. Pretending you did X when you didn't is not.
- **Do not redistribute the provided dataset or test footage.** Treat both as confidential. Your repository should contain code only, not data.
- If something is ambiguous, make a reasonable assumption, document it, and proceed. We will discuss your assumptions in the follow-up.

---

6. Follow-up interview

After we review your submission, we'll schedule a 60-minute technical conversation. Expect to:

- Walk us through your architecture and the tradeoffs you made
- Defend your model choice against alternatives we'll propose
- Defend your FP-reduction approach against alternatives we'll propose
- Discuss what changes if the deployment target moves from Orin Nano to Orin NX, or if the latency budget halves
- Talk through how you would extend the evaluation and dataset to be deployment-grade

Good luck. We're looking forward to seeing how you think.